

AI Powered Vehicle Recommendation Chatbot Using RAG (Eloqwent AI assistant)

PROJECT REPORT

Project	AI Powered Vehicle Recommendation Chatbot Using RAG
Application	Eloqwent — AI Mobility Assistant
Architecture	RAG + Vector Search + LLM
Vector Database	DataStax Astra DB
AI API	OpenRouter
Framework	Next.js 16.3.0
Runtime	Node.js

Prepared from the project implementation and development details supplied for this system.

1. Abstract

The Tata Motors RAG Chatbot is an AI-powered question-answering application designed to answer questions using a controlled Tata Motors knowledge base. It combines Retrieval-Augmented Generation (RAG), document processing, text chunking, vector embeddings, vector similarity search, DataStax Astra DB, and an OpenRouter-hosted language model.

The user-facing application is named **Eloqwent** and provides a conversational interface with chat history, suggested questions, authentication-related components, and document upload. The system is designed so additional information can be added by ingesting more supported documents.

2. Introduction

Large language models can produce fluent answers, but a general model may not contain organization-specific information or may generate unsupported statements. RAG addresses this by retrieving relevant external information before generating an answer.

This project applies RAG to Tata Motors information. The ingestion side prepares source material for semantic search, while the question-answering side retrieves relevant chunks and supplies them to the language model as context.

3. Problem Statement

- Provide a conversational interface for Tata Motors-related questions.
- Use project-specific documents and web information as the knowledge source.
- Reduce unsupported answers by grounding generation in retrieved context.
- Allow the knowledge base to be expanded with additional documents.
- Store searchable vector representations in a scalable vector database.

4. Objectives

- Build an end-to-end RAG chatbot.
- Extract information from PDFs, JSONL data and configured websites.
- Split documents into retrieval-friendly overlapping chunks.
- Generate embeddings for document chunks and user questions.
- Store vectors and metadata in DataStax Astra DB.
- Retrieve relevant chunks using vector similarity.
- Generate concise answers using an LLM.
- Provide a practical web interface for conversation and document upload.

5. Proposed Solution

The system follows the pipeline: **Sources** → **Text Extraction** → **Cleaning** → **Chunking** → **Embedding** → **Astra DB** → **Similarity Retrieval** → **Context Construction** → **LLM** → **Answer**.

This architecture separates the knowledge layer from response generation. The vector database stores the searchable knowledge, while the language model generates an answer using retrieved context.

6. System Architecture

Frontend: Next.js chat interface, conversations, prompts and document selection.

Backend: Next.js API routes for chat and document upload.

Ingestion: scripts/loadDb.ts loads PDFs, JSONL files and configured websites.

Processing: text cleaning and RecursiveCharacterTextSplitter.

Embedding: OpenRouter embedding model.

Storage: DataStax Astra DB vector collection.

Retrieval: top-K vector similarity search.

Generation: OpenRouter chat model using a grounded system prompt.

7. Technology Stack

Next.js 16.3.0 / React 19.2.8: web application and UI.

TypeScript: application and server-side typing.

DataStax Astra DB: vector database.

OpenRouter + OpenAI SDK: embedding and chat-model access.

LangChain Text Splitters: recursive chunking.

Puppeteer: website extraction.

Mammoth: DOCX extraction.

XLSX: spreadsheet extraction.

unpdf: PDF text extraction in the upload route.

dotenv: environment configuration.

NextAuth: authentication/session support.

8. Data Ingestion Pipeline

The main bulk-ingestion script is **scripts/loadDb.ts**. It scans the PDF and JSONL data directories and processes configured website sources.

PDF files are read page-by-page. JSONL files are parsed line-by-line. Websites are opened with Puppeteer and body text is collected. Text is cleaned, empty documents are removed, and duplicate

documents are filtered.

Documents are split using RecursiveCharacterTextSplitter with a chunk size of 800 and overlap of 150. Each chunk receives a unique ID and metadata. Embeddings are generated and records are inserted into Astra DB in batches of 20.

9. Document Upload and Processing

The upload API accepts multipart form data. It validates the uploaded file, extracts readable text, rejects empty results, splits the text into chunks, generates embeddings and inserts the resulting vectors into Astra DB.

The supplied upload implementation supports PDF, DOC/DOCX, XLS/XLSX, TXT, Markdown, JSON and CSV. PDF extraction currently uses unpdf, DOC/DOCX uses Mammoth, and spreadsheets use XLSX.

10. Embedding Generation

The embedding layer exposes **createEmbedding()** for one text and **createEmbeddings()** for multiple texts. It uses the OpenAI SDK configured with OpenRouter.

The configured default embedding model is **nvidia/nemotron-3-embed-1b:free**, while the model can also be selected through the EMBEDDING_MODEL environment variable. Embeddings convert text into numerical vectors suitable for semantic similarity search.

11. Astra DB Vector Storage

The Astra helper initializes a Data API client using the configured application token and API endpoint, then selects the configured keyspace and collection.

Stored records contain chunk text, source, category, type, a unique identifier and the **\$vector** embedding. The recreated database configuration was intended to retain the project's required embedding dimension and cosine similarity settings; the collection definition must remain compatible with the embedding model output.

12. Retrieval-Augmented Generation

The chat API receives the question, creates a question embedding, and performs vector similarity search against Astra DB. The current implementation requests the top **5** results.

The retrieved chunks are combined into a context containing document information and text. A system prompt instructs the LLM to answer only from the supplied context, avoid invented facts, and state when the information cannot be found.

13. Chat Application Workflow

- User opens Eloqwent.

- A conversation is created or restored from localStorage.
- User asks a question or chooses a suggested question.
- Frontend sends the question to /api/chat.
- Question is converted to an embedding.
- Astra DB returns relevant chunks.
- Chunks are placed into the RAG context.
- OpenRouter generates the answer.
- Assistant response is shown in the chat.
- Conversation state is saved locally.

14. Project File Structure

app/chat/page.tsx: main chat interface and interaction logic.

app/chat/chat.css: chat styling.

app/api/chat/route.ts: question embedding, retrieval and LLM generation.

app/api/upload/route.ts: document extraction, chunking, embedding and insertion.

components/: Navbar, UserMenu and prompt-suggestion components.

lib/openrouter.ts: OpenRouter client and model configuration.

lib/embeddings.ts: embedding functions.

lib/astra.ts: Astra DB client and collection access.

scripts/loadDb.ts: bulk ingestion pipeline.

data/pdf/: PDF sources.

data/jsonl/: JSONL datasets.

data/urls.ts: website sources.

.env.local: private credentials/configuration.

package.json: dependencies and npm scripts.

15. Key Implementation Details

Chunking: the bulk loader uses chunkSize 800 and chunkOverlap 150. The upload route follows the same general overlapping-chunk approach.

Batching: the bulk loader uploads vector records in batches of 20.

Chat retrieval: TOP_K is set to 5.

Models: EMBEDDING_MODEL and LLM_MODEL are configurable environment variables.

Metadata: source, category, type and identifiers are stored alongside chunks.

Grounding: the system prompt explicitly prohibits unsupported facts.

16. Testing and Validation

The project was tested during development with text uploads and the chat response path. PDF extraction produced compatibility problems with earlier PDF.js worker configurations. The upload route was subsequently moved to unpdf for PDF extraction.

Recommended validation is: run the development server, upload a TXT file, confirm insertion in Astra DB, upload a text-based PDF, inspect extraction, run bulk ingestion after adding new files, and ask questions whose answers are present in the knowledge base.

17. Limitations

- Answer quality depends on source quality and coverage.
- PDF extraction depends on parser compatibility and document structure.
- Scanned/image-only PDFs require OCR.
- Retrieval currently uses a fixed top-K of 5.
- Conversation history is stored in browser localStorage.
- No advanced reranking stage is described in the supplied implementation.
- Embedding API availability and rate limits can affect ingestion speed.

18. Future Enhancements

- Add OCR for scanned PDFs.
- Add page numbers and richer source metadata.
- Add reranking after vector retrieval.
- Add similarity thresholds and configurable top-K.
- Add document versioning and deletion/update operations.
- Persist conversations server-side.
- Add streaming LLM responses.
- Add retrieval and answer-quality evaluation metrics.
- Automate periodic ingestion of updated sources.
- Display source citations and document previews in the UI.

19. Security and Deployment Considerations

- Never commit API keys or Astra credentials to GitHub.
- Keep .env.local in .gitignore.
- Retain file-size and file-type validation.

- Use HTTPS and restricted production credentials.
- Protect upload functionality with authentication where required.
- Keep document parsing dependencies maintained.
- Review npm audit findings before production deployment.

20. Conclusion

The Tata Motors RAG Chatbot demonstrates a practical end-to-end Retrieval-Augmented Generation system. It combines a Next.js frontend, document ingestion, text chunking, OpenRouter embeddings, Astra DB vector search and grounded LLM generation.

The architecture is extensible: additional supported information can be added to the data directories and ingested into the configured collection. Separating ingestion, retrieval and generation provides a clear foundation for future improvements such as OCR, reranking, evaluation, persistent conversations and richer source citations.

21. References

Next.js — <https://nextjs.org/docs>

DataStax Astra DB — <https://docs.datastax.com/en/astra-db-serverless/>

OpenRouter — <https://openrouter.ai/docs>

LangChain — <https://docs.langchain.com/>

Puppeteer — <https://pptr.dev/>

Mammoth — <https://github.com/mwilliamson/mammoth.js>

SheetJS — <https://docs.sheetjs.com/>

unpdf — <https://github.com/unjs/unpdf>

Report basis: This report is based on the project code, configuration and development details supplied in the conversation. Exact runtime performance metrics were not claimed where they were not provided.